

Método de Alineación Progresiva

Johan Fabricio Lizarve Mamani
Edilson Bonet Mamani Yucra
Josue Samuel Philco Puma

11 de junio de 2026

1. Introducción

La alineación de múltiples secuencias (MSA, por sus siglas en inglés *Multiple Sequence Alignment*) es una de las tareas fundamentales en bioinformática. Su objetivo es identificar regiones de similitud entre tres o más secuencias biológicas —ya sean nucleótidos o aminoácidos— con el fin de inferir relaciones evolutivas, predecir estructura y función, o detectar motivos conservados.

En este laboratorio se implementó el método de **alineación progresiva**, una estrategia heurística ampliamente utilizada que descompone el problema en una serie de alineaciones por pares. El proceso se guía mediante un **árbol guía** construido a partir de una **matriz de distancias** calculada con el algoritmo de **Needleman-Wunsch**. Las secuencias o grupos de secuencias se van fusionando progresivamente según el árbol guía, utilizando una versión modificada del algoritmo que soporta el carácter especial X para tratar columnas de gaps.

2. Ejercicios

Para poder realizar la **alineación progresiva** en varias secuencias debemos seguir una serie de pasos para poder llegar al resultado esperado, para ello debemos ver que pasos vamos a realizar para la implementación. Primero debemos hablar del árbol guía, este debe obtenerse siguiendo estos pasos:

1. Utilizar estructuras de datos para la lectura de los archivos Fasta.
2. Vamos a empezar con una matriz de distancia mediante el algoritmo de **Needle-Wunsch**.
3. Seleccionar las 2 secuencias que tengan la menor distancia y crear un nuevo agrupamiento.
4. Calcular nuevamente la matriz de distancia pero teniendo consideración en el método **UPGMA**.
5. Y repetir este proceso hasta llegar al árbol guía.

Ahora que sabemos los pasos del árbol guía también debemos cumplir unos pasos restantes los cuáles son:

1. Se debe fusionar las secuencias según el árbol guía.
2. Para calcular las distancias de una secuencia a un grupo debemos utilizar **Needle-Wunsch** pero modificarlo para que tenga soporte con los valores **X**.
3. Se repite el proceso hasta tener las secuencias alineadas de la misma longitud.
4. Por último, en las secuencias reemplazar los valores **X** por - para mostrar las secuencias al final.

Ahora si con los pasos numerados vamos por el desarrollo de esta implementación.

2.1. Lectura de Archivos Fasta

Como ya es conocido, debemos leer los distintos archivos FASTA que vamos a utilizar para probarlos, para ello utilizamos una estructura que va a almacenar su encabezado (identificador) y la secuencia que es lo más importante, con ello ya cumplimos con la parte de la lectura de los archivos correspondientes.

Listing 1: Lectura de los archivos FASTA

```
1 struct FastaSequence {
2     string header;
3     string sequence;
4 };
5
6 vector<FastaSequence> readFileFasta(const string &filename) {
7     ifstream file_fasta(filename);
8     vector<FastaSequence> sequences;
9
10    if (!file_fasta.is_open()) {
11        cerr << "Error opening file: " << filename << endl;
12        return sequences;
13    }
14
15    string line;
16    FastaSequence current_sequence;
17
18    while(getline(file_fasta, line)) {
19        if (line.empty()) {
20            continue;
21        }
22
23        if (line[0] == '>') {
24            if (!current_sequence.header.empty()) {
25                sequences.push_back(current_sequence);
26                current_sequence = FastaSequence();
27            }
28            current_sequence.header = line.substr(1);
29        }
30        else {
31            current_sequence.sequence += line;
32        }
33    }
34
35    if (!current_sequence.header.empty()) {
36        sequences.push_back(current_sequence);
37    }
38
39    return sequences;
40 }
```

La lógica es sencilla: cuando se encuentra una línea que comienza con >, se guarda la secuencia acumulada (si existe) y se inicia una nueva. Las líneas restantes se concatenan a la secuencia actual.

2.2. Algoritmo Needle-Wunsch

Ahora vamos con el algoritmo **Needle-Wunsch**, el cuál va a ser utilizado para poder calcular la matriz de distancia en las secuencias. Pero a diferencia de la práctica anterior, este algoritmo no solo nos va a servir para la matriz de distancia inicial, sino que también debe realizar el cálculo de la matriz de distancia considerando la operación de **UPGMA**. El algoritmo inicial se ve de esta forma (también hace uso de una estructura que almacena las secuencias alineadas y el score encontrado).

Listing 2: Needle-Wunsch para el Alineamiento Global

```
1 struct AlignmentResult {
2     string aligned_seq1;
3     string aligned_seq2;
4     int score;
5 };
6
7 AlignmentResult needle_wunsch(const string &seq1, const string &seq2, int match,
8     int mismatch, int gap) {
```

```

8      int n = seq1.size();
9      int m = seq2.size();
10
11     vector<vector<int>> score_matrix(n + 1, vector<int>(m + 1, 0));
12
13     for (int i = 0; i <= n; i++) {
14         score_matrix[i][0] = i * gap;
15     }
16     for (int j = 0; j <= m; j++) {
17         score_matrix[0][j] = j * gap;
18     }
19
20     for (int i = 1; i <= n; i++) {
21         for (int j = 1; j <= m; j++) {
22             int score_diagonal = score_matrix[i - 1][j - 1] + (seq1[i - 1] == seq2[
                j - 1] ? match : mismatch);
23             int score_up = score_matrix[i - 1][j] + gap;
24             int score_left = score_matrix[i][j - 1] + gap;
25
26             score_matrix[i][j] = max({score_diagonal, score_up, score_left});
27         }
28     }
29
30     string aligned_seq1, aligned_seq2;
31     int i = n, j = m;
32
33     while (i > 0 || j > 0) {
34         if (i > 0 && j > 0) {
35             int score_diagonal = score_matrix[i - 1][j - 1] + (seq1[i - 1] == seq2[
                j - 1] ? match : mismatch);
36             int score_up = score_matrix[i - 1][j] + gap;
37             int score_left = score_matrix[i][j - 1] + gap;
38
39             if (score_matrix[i][j] == score_diagonal) {
40                 aligned_seq1 += seq1[i - 1];
41                 aligned_seq2 += seq2[j - 1];
42                 i--;
43                 j--;
44             }
45             else if (score_matrix[i][j] == score_up) {
46                 aligned_seq1 += seq1[i - 1];
47                 aligned_seq2 += '-';
48                 i--;
49             }
50             else {
51                 aligned_seq1 += '-';
52                 aligned_seq2 += seq2[j - 1];
53                 j--;
54             }
55         }
56         else if (i > 0) {
57             aligned_seq1 += seq1[i - 1];
58             aligned_seq2 += '-';
59             i--;
60         }
61         else {
62             aligned_seq1 += '-';
63             aligned_seq2 += seq2[j - 1];
64             j--;
65         }
66     }
67
68     reverse(aligned_seq1.begin(), aligned_seq1.end());

```

```

69     reverse(aligned_seq2.begin(), aligned_seq2.end());
70
71     return {aligned_seq1, aligned_seq2, score_matrix[n][m]};
72 }

```

Como tal, este primer algoritmo nos servirá para calcular la matriz de distancia inicial para luego poder crear un nuevo agrupamiento. Pero al momento en que usemos **UPGMA** ya no vamos a tener que procesar con este algoritmo, sino que vamos a utilizar una adaptación del mismo algoritmo pero utilizando un **Modelo de Costo Unitario** el cuál es:

$$s(a, a) = 0, \quad s(a, b) = 1, \quad s(X, a) = s(X, b) = 0$$

Entonces el algoritmo adaptado a ese modelo quedaría de la siguiente manera

Listing 3: Needle-Wunsch adaptado al costo unitario de UPGMA

```

1  AlignmentResult needle_wunsch(const string &seq1, const string &seq2, int match,
2     int mismatch, int gap, bool use_model_x) {
3     int n = seq1.size();
4     int m = seq2.size();
5
6     vector<vector<int>> score_matrix(n + 1, vector<int>(m + 1, 0));
7
8     for (int i = 0; i <= n; i++) {
9         score_matrix[i][0] = i * (use_model_x ? 1 : gap);
10    }
11    for (int j = 0; j <= m; j++) {
12        score_matrix[0][j] = j * (use_model_x ? 1 : gap);
13    }
14
15    for (int i = 1; i <= n; i++) {
16        for (int j = 1; j <= m; j++) {
17            int cost = use_model_x ? ((seq1[i-1] == 'X' || seq2[j-1] == 'X') ? 0 :
18                (seq1[i-1] == seq2[j-1] ? 0 : 1)) : (seq1[i-1] == seq2[j-1] ? match
19                : mismatch);
20
21            int score_diagonal = score_matrix[i - 1][j - 1] + cost;
22            int score_up = score_matrix[i - 1][j] + (use_model_x ? 1 : gap);
23            int score_left = score_matrix[i][j - 1] + (use_model_x ? 1 : gap);
24
25            score_matrix[i][j] = use_model_x ? min({score_diagonal, score_up,
26                score_left}) : max({score_diagonal, score_up, score_left});
27        }
28    }
29
30    string aligned_seq1, aligned_seq2;
31    int i = n, j = m;
32
33    while (i > 0 || j > 0) {
34        if (i > 0 && j > 0) {
35            int cost = use_model_x ? ((seq1[i-1] == 'X' || seq2[j-1] == 'X') ? 0 :
36                (seq1[i-1] == seq2[j-1] ? 0 : 1)) : (seq1[i-1] == seq2[j-1] ? match
37                : mismatch);
38
39            int score_diagonal = score_matrix[i - 1][j - 1] + cost;
40            int score_up = score_matrix[i - 1][j] + (use_model_x ? 1 : gap);
41            int score_left = score_matrix[i][j - 1] + (use_model_x ? 1 : gap);
42
43            if (score_matrix[i][j] == score_diagonal) {
44                aligned_seq1 += seq1[i - 1];
45                aligned_seq2 += seq2[j - 1];
46                i--;
47                j--;
48            }
49        }
50    }
51 }

```

```

43         else if (score_matrix[i][j] == score_up) {
44             aligned_seq1 += seq1[i - 1];
45             aligned_seq2 += '-';
46             i--;
47         }
48         else {
49             aligned_seq1 += '-';
50             aligned_seq2 += seq2[j - 1];
51             j--;
52         }
53     }
54     else if (i > 0) {
55         aligned_seq1 += seq1[i - 1];
56         aligned_seq2 += '-';
57         i--;
58     }
59     else {
60         aligned_seq1 += '-';
61         aligned_seq2 += seq2[j - 1];
62         j--;
63     }
64 }
65
66 reverse(aligned_seq1.begin(), aligned_seq1.end());
67 reverse(aligned_seq2.begin(), aligned_seq2.end());
68
69 return {aligned_seq1, aligned_seq2, score_matrix[n][m]};
70 }

```

2.3. Calculando la Matriz de Distancia

Ahora que tenemos el algoritmo **Needle-Wunsch** implementado vamos a pasar con el cálculo de la matriz de distancia. Este lo vamos a hacer de la siguiente manera.

1. Vamos a crear una matriz de distancias de tamaño $n \times n$, donde n representa la cantidad de secuencias que se están procesando. Inicialmente, todos sus valores se establecen en 0.
2. Se calcula el **self-score** de cada secuencia alineándola consigo misma con el algoritmo Needleman-Wunsch. Esto va a representar el puntaje máximo esperado para dicha secuencia.
3. Para cada par de secuencias (i, j) , se ejecuta Needleman-Wunsch para obtener el puntaje de alineamiento entre ambas secuencias.
4. Se debe obtener el mayor valor entre los **self-scores** de las dos secuencias;

$$\max(S_{ii}, S_{jj})$$

donde S_{ii} y S_{jj} corresponden a los puntajes de las secuencias alineadas consigo mismas.

5. Con el puntaje de alineamiento entre las secuencias se va a normalizar utilizando la siguiente expresión:

$$\frac{S_{ij}}{\max(S_{ii}, S_{jj})}$$

donde S_{ij} es el puntaje obtenido al alinear las secuencias i y j . Y la distancia entre ambas secuencias se calcula como:

$$d(i, j) = 1 - \frac{S_{ij}}{\max(S_{ii}, S_{jj})}$$

6. El valor calculado se almacena en las posiciones (i, j) y (j, i) de la matriz, ya que la distancia entre dos secuencias es simétrica.
7. La diagonal principal permanece con valor 0 ya que dos secuencias iguales no van a compararse.

8. Una vez procesados todos los pares de secuencias, la matriz resultante va a contener las distancias normalizadas que posteriormente van a ser utilizadas para construir el **Árbol Guía**.

Ahora con estos pasos numerados la función para calcular la matriz de distancia es la siguiente.

Listing 4: Implementación para el cálculo de la matriz de distancia inicial

```

1 vector<vector<double>> computeDistanceMatrix(const vector<FastaSequence> &sequences
  ) {
2   int n = sequences.size();
3   vector<vector<double>> distance_matrix(n, vector<double>(n, 0.0));
4   vector<int> self_scores(n, 0);
5
6   for (int i = 0; i < n; i++) {
7     self_scores[i] = needle_wunsch(sequences[i].sequence, sequences[i].sequence
      , MATCH, MISMATCH, GAP, false).score;
8   }
9
10  for (int i = 0; i < n; i++) {
11    for (int j = i + 1; j < n; j++) {
12      AlignmentResult result = needle_wunsch(sequences[i].sequence, sequences
        [j].sequence, MATCH, MISMATCH, GAP, false);
13      double distance = 1.0 - (static_cast<double>(result.score) / max(
        self_scores[i], self_scores[j]));
14
15      distance_matrix[i][j] = distance;
16      distance_matrix[j][i] = distance;
17    }
18  }
19
20  return distance_matrix;
21 }
```

Ahora que tenemos la matriz de distancia inicial podemos empezar con la función que hará el proceso de **UPGMA** correspondiente para poder obtener el **Árbol Guía**.

2.4. Método UPGMA

Con la matriz inicial obtenida, pasamos con el método **UPGMA** el cuál es un algoritmo jerárquico que va a construir nuestro **Árbol Guía**, para ello vamos a seguir estos pasos:

1. Se identifican los dos grupos C_i y C_j con la menor distancia.
2. Se fusionan en un nuevo grupo C_{ij} .
3. Se recalcula la distancia del nuevo grupo a todos los demás usando la media ponderada:

$$d(C_{ij}, C_k) = \frac{|C_i| \cdot d(C_i, C_k) + |C_j| \cdot d(C_j, C_k)}{|C_i| + |C_j|}$$

4. Se actualiza el tamaño del nuevo grupo:

$$|C_{ij}| = |C_i| + |C_j|$$

Y el grupo C_j se marca como inactivo, ya que ahora forma parte del nuevo grupo fusionado.

5. Se repite hasta obtener un único grupo.

Ahora con los pasos enumerados, la implementación para el método **UPGMA** sería el siguiente.

Listing 5: Implementación del método UPGMA

```

1 vector<GuideTree> computeUPGMA(vector<vector<double>> &distance_matrix, int n) {
2   vector<GuideTree> guide_tree;
3   vector<int> cluster_sizes(n, 1);
```

```
4     vector<bool> active(n, true);
5
6     for (int step = 0; step < n - 1; step++) {
7         int min_i = -1, min_j = -1;
8         double min_distance = numeric_limits<double>::max();
9
10        for (int i = 0; i < n; i++) {
11            for (int j = i + 1; j < n; j++) {
12                if (active[i] && active[j] && distance_matrix[i][j] < min_distance)
13                {
14                    min_distance = distance_matrix[i][j];
15                    min_i = i;
16                    min_j = j;
17                }
18            }
19
20            guide_tree.push_back({min_i, min_j});
21
22            for (int k = 0; k < n; k++) {
23                if (!active[k] || k == min_i || k == min_j) {
24                    continue;
25                }
26                distance_matrix[min_i][k] = distance_matrix[k][min_i] =
27                    (cluster_sizes[min_i] * distance_matrix[min_i][k] +
28                     cluster_sizes[min_j] * distance_matrix[min_j][k]) /
29                    (cluster_sizes[min_i] + cluster_sizes[min_j]);
30            }
31
32            cluster_sizes[min_i] += cluster_sizes[min_j];
33            active[min_j] = false;
34        }
35
36        return guide_tree;
37    }
```

Ahora ya estamos cerca de poder cumplir con la construcción y obtener nuestro árbol guía, solo nos falta realizar la fusión de grupos de las secuencias.

2.5. Fusión de los Grupos de Secuencias

Ahora pasamos con la etapa final de todo el proceso, esta parte la vamos a dividir en 3 partes donde uno es la **aplicación de gaps**, **unión de los grupos** y **alineamiento múltiple progresivo**

2.5.1. Propagación de huecos en un grupo

Cuando dos grupos van a fusionarse, primero se alinean sus secuencias representativas utilizando el algoritmo Needleman-Wunsch. Como resultado, pueden introducirse nuevos huecos (–) en las secuencias representativas alineadas.

Para mantener la consistencia del alineamiento, dichos huecos deben propagarse a todas las secuencias pertenecientes al grupo correspondiente.

1. Si el carácter actual es un hueco (–), se inserta un hueco en la nueva secuencia.
2. En caso contrario, se copia el siguiente carácter de la secuencia original.
3. El proceso se repite hasta reconstruir la secuencia con la misma estructura de huecos que la secuencia representativa alineada.

De esta manera, todas las secuencias del grupo conservan la misma longitud y posición relativa de los huecos.

Listing 6: Propagación de los gaps

```
1 vector<string> applyGaps(const vector<string> &group, const string &aligned_rap) {
2     vector<string> result;
3
4     for (const string &sequence : group) {
5         string new_sequence = "";
6         int sequence_index = 0;
7
8         for (char c : aligned_rap) {
9             if (c == '-') {
10                 new_sequence += '-';
11             }
12             else {
13                 new_sequence += sequence[sequence_index];
14                 sequence_index++;
15             }
16         }
17         result.push_back(new_sequence);
18     }
19
20     return result;
21 }
```

2.5.2. Fusión de grupos

Para fusionar dos grupos alineados, se selecciona una secuencia representativa de cada uno de ellos.

1. Se alinean ambas secuencias representativas mediante Needleman-Wunsch.
2. Los huecos obtenidos en cada secuencia representativa se propagan al resto de secuencias de su respectivo grupo.
3. Los dos grupos alineados se concatenan para formar un único grupo.

El resultado es un nuevo grupo que contiene todas las secuencias de ambos conjuntos con una estructura de alineamiento compatible.

Listing 7: Fusión de los grupos

```
1 vector<string> mergeGroups(const vector<string> &group1, const vector<string> &
2     group2) {
3     string rep_a = group1[0];
4     string rep_b = group2[0];
5
6     AlignmentResult result = needle_wunsch(rep_a, rep_b, 0, 1, 1, true);
7
8     vector<string> aligned_group1 = applyGaps(group1, result.aligned_seq1);
9     vector<string> aligned_group2 = applyGaps(group2, result.aligned_seq2);
10
11     vector<string> merged;
12     merged.insert(merged.end(), aligned_group1.begin(), aligned_group1.end());
13     merged.insert(merged.end(), aligned_group2.begin(), aligned_group2.end());
14
15     return merged;
16 }
```

2.5.3. Alineamiento múltiple progresivo

El alineamiento múltiple de secuencias se realiza siguiendo el orden de fusiones establecido por el árbol guía generado previamente mediante UPGMA.

Inicialmente, cada secuencia forma un grupo independiente. Luego, para cada unión especificada en el árbol guía:

1. Se seleccionan los dos grupos indicados.
2. Se realiza la fusión de ambos grupos mediante el procedimiento descrito anteriormente.
3. El grupo resultante reemplaza al primer grupo involucrado en la fusión.
4. El segundo grupo se marca como vacío al haber sido incorporado al nuevo conjunto.

Este proceso continúa hasta completar todas las fusiones del árbol guía. Finalmente, el único grupo no vacío contiene el alineamiento múltiple final de todas las secuencias de entrada.

Listing 8: Alineamiento Múltiple Progresivo

```
1 vector<string> performMSA(const vector<FastaSequence> &sequences, const vector<
  GuideTree> &guide_tree) {
2     vector<vector<string>> groups;
3     for (const auto &seq : sequences) {
4         groups.push_back({seq.sequence});
5     }
6
7     for (const auto &merge : guide_tree) {
8         vector<string> merged_groups = mergeGroups(groups[merge.sequence_index1],
9             groups[merge.sequence_index2]);
10        groups[merge.sequence_index1] = merged_groups;
11        groups[merge.sequence_index2] = {};
12    }
13
14    for (const auto &group : groups) {
15        if (!group.empty()) {
16            return group;
17        }
18    }
19    return {};
20 }
```

2.6. Limpieza de las secuencias

Una vez completada la alineación, todos los X que se usaron como marcadores de gap durante las fusiones intermedias son reemplazados por el carácter estándar '-'.

Listing 9: Limpieza de caracteres X

```
1 vector<string> replaceX(const vector<string> &aligned_sequences) {
2     vector<string> result;
3     for (const string &seq : aligned_sequences) {
4         string clean = "";
5         for (char c : seq)
6             clean += (c == 'X') ? '-' : c;
7         result.push_back(clean);
8     }
9     return result;
10 }
```

Ahora que tenemos todos los pasos implementados pasemos con la parte de resultados del **Método de Alineación Progresiva**.

3. Probando la Alineación Progresiva

Ahora con la implementación completa, vamos a probar con dos conjuntos de secuencias: un conjunto de 4 secuencias de una práctica en clase y el otro conjunto de 5 secuencias proporcionadas para el laboratorio.

Secuencias de la práctica

En clase se realizó este método de forma escrita, ahora con la implementación debemos ver si obtenemos los mismos resultados, las secuencias que utilizaremos con las siguientes.

```
S1 = ATTGGCACCA
S2 = ATTTGGACCA
S3 = TGGTTCCA
S4 = ATTCCACCAC
```

Entonces vamos a ver como es que realizo la implementación con estos pasos:

■ Matriz de distancia inicial

```
=== Initial Distance Matrix ===
0.0000 0.4000 1.0000 0.7000
0.4000 0.0000 1.2000 0.9000
1.0000 1.2000 0.0000 1.4000
0.7000 0.9000 1.4000 0.0000
```

■ Método UPGMA para el Árbol Guía

```
=== UPGMA Guide Tree ===

Step 1: merge seq 0 (seq1) with seq 1 (seq2)
distance: 0.40

Updated Distance Matrix:
0.0000 1.1000 0.8000
1.1000 0.0000 1.4000
0.8000 1.4000 0.0000
Step 2: merge seq 0 (seq1) with seq 3 (seq4)
distance: 0.80

Updated Distance Matrix:
0.0000 1.2000
1.2000 0.0000
Step 3: merge seq 0 (seq1) with seq 2 (seq3)
distance: 1.20

Updated Distance Matrix:
0.0000
```

■ Alineamiento Múltiple progresivo

```
=== Final MSA ===
>seq1  ATTGGCACCA-
>seq2  ATTTGGACCA-
>seq3  ATT-CCACCA-
>seq4  -TGGTTCCA-
```

El resultado como tal se exporta a un archivo de texto donde también guardamos como es el **Árbol Guía**.

```

C++ ProgressiveAlignment.cpp | results.txt | .gitignore | GuideTree.py
Unidad 2 > results.txt
1 Loaded 4 sequences.
2
3 === Initial Distance Matrix ===
4 0.0000 0.4000 1.0000 0.7000
5 0.4000 0.0000 1.2000 0.9000
6 1.0000 1.2000 0.0000 1.4000
7 0.7000 0.9000 1.4000 0.0000
8
9 === UPGMA Guide Tree ===
10
11 Step 1: merge seq 0 (seq1) with seq 1 (seq2) | distance: 0.40
12 Updated Distance Matrix:
13 0.0000 1.1000 0.8000
14 1.1000 0.0000 1.4000
15 0.8000 1.4000 0.0000
16
17 Step 2: merge seq 0 (seq1) with seq 3 (seq4) | distance: 0.80
18 Updated Distance Matrix:
19 0.0000 1.2000
20 1.2000 0.0000
21
22 Step 3: merge seq 0 (seq1) with seq 2 (seq3) | distance: 1.20
23 Updated Distance Matrix:
24 0.0000
25
26 === Final MSA ===
27 >seq1
28 ATTGGACCA-
29 >seq2
30 ATTGGACCA-
31 >seq3
32 ATT-CCACCA-
33 >seq4
34 --TGGTTCCA-
  
```

Figura 1: Resultado con las secuencias de la práctica en clase

Y por último vamos a ver como se ha creado el **Árbol Guía** correspondiente, esto lo haremos con ayuda de Python.

```

Guide Tree Data
Sequences
0 seq1
1 seq2
2 seq3
3 seq4
Merges 0 1 0.40
0 3 0.70
0 2 1.00
end
  
```

UPGMA Guide Tree

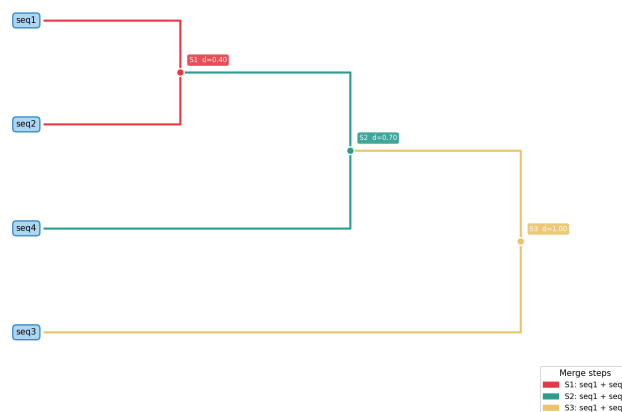


Figura 2: Árbol Guía obtenido

Secuencias proporcionadas

Ahora pasamos con las secuencias proporcionadas en este laboratorio, siendo las siguientes secuencias.

- **NR_024570.1:** *Escherichia coli* strain U 5/41, 16S ribosomal RNA, partial sequence.
- **NR_114475.1:** *Oceanimonas doudoroffii* strain ATCC 27123, 16S ribosomal RNA, partial sequence.
- **NR_026331.1:** *Shigella flexneri* strain ATCC 29903, 16S ribosomal RNA, partial sequence.
- **NR_117686.1:** *Klebsiella pneumoniae* strain DSM 30104, 16S ribosomal RNA, partial sequence.
- **NR_118568.1:** *Enterobacter cloacae* strain ATCC 13047, 16S ribosomal RNA, partial sequence.

Ahora vamos a ver el resultado que obtenemos con la implementación.

■ Matriz de distancia inicial

```
=== Initial Distance Matrix ===  
0.0000 0.3179 0.1223 0.2471 0.2269  
0.3179 0.0000 0.2715 0.3542 0.3813  
0.1223 0.2715 0.0000 0.1601 0.2849  
0.2471 0.3542 0.1601 0.0000 0.3471  
0.2269 0.3813 0.2849 0.3471 0.0000
```

■ Método UPGMA para el Árbol Guía

```
=== UPGMA Guide Tree ===  
  
Step 1: merge seq 0 (NR_024570.1) with seq 2 (NR_026331.1)  
distance: 0.12  
Updated Distance Matrix:  
0.0000 0.2947 0.2036 0.2559  
0.2947 0.0000 0.3542 0.3813  
0.2036 0.3542 0.0000 0.3471  
0.2559 0.3813 0.3471 0.0000  
  
Step 2: merge seq 0 (NR_024570.1) with seq 3 (NR_117686.1)  
distance: 0.20  
Updated Distance Matrix:  
0.0000 0.3145 0.2863  
0.3145 0.0000 0.3813  
0.2863 0.3813 0.0000  
  
Step 3: merge seq 0 (NR_024570.1) with seq 4 (NR_118568.1)  
distance: 0.29  
Updated Distance Matrix:  
0.0000 0.3312  
0.3312 0.0000  
  
Step 4: merge seq 0 (NR_024570.1) with seq 1 (NR_114475.1)  
distance: 0.33  
Updated Distance Matrix:  
0.0000
```


Guide Tree Data

Sequences

```

0 NR_024570.1 (Escherichia coli)
1 NR_114475.1 (Oceanimonas doudoroffii)
2 NR_026331.1 (Shigella flexneri)
3 NR_117686.1 (Klebsiella pneumoniae)
4 NR_118568.1 (Enterobacter cloacae)

```

Merges

```

0 2 0.12
0 3 0.25
0 4 0.23
0 1 0.32

```

end

UPGMA Guide Tree

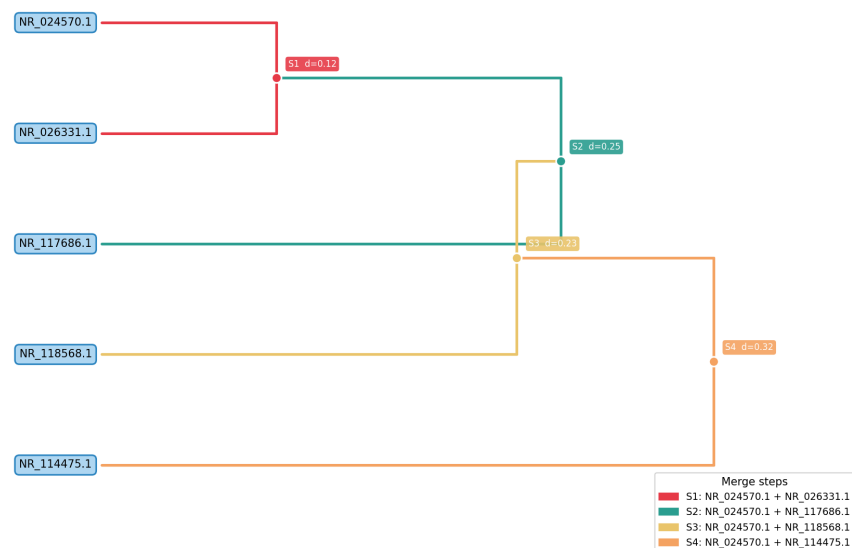


Figura 5: Árbol Guía obtenido

4. Programa completo

Esta implementación se desarrollo en C++ para el **Método de Alineación Progresiva** y Python para visualizar el **Árbol Guía**.

4.1. Código C++

Listing 10: Programa C++

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <fstream>
5 #include <unordered_map>
6 #include <algorithm>
7 #include <limits>

```

```
8 #include <iomanip>
9
10 using namespace std;
11
12 const int MATCH = 1;
13 const int MISMATCH = -1;
14 const int GAP = -2;
15
16 struct FastaSequence {
17     string header;
18     string sequence;
19 };
20
21 struct AlignmentResult {
22     string aligned_seq1;
23     string aligned_seq2;
24     int score;
25 };
26
27 struct GuideTree {
28     int sequence_index1;
29     int sequence_index2;
30     double distance;
31     int size;
32 };
33
34 vector<FastaSequence> readFileFasta(const string &filename) {
35     ifstream file_fasta(filename);
36     vector<FastaSequence> sequences;
37
38     if (!file_fasta.is_open()) {
39         cerr << "Error opening file: " << filename << endl;
40         return sequences;
41     }
42
43     string line;
44     FastaSequence current_sequence;
45
46     while(getline(file_fasta, line)) {
47         if (line.empty()) {
48             continue;
49         }
50
51         if (line[0] == '>') {
52             if (!current_sequence.header.empty()) {
53                 sequences.push_back(current_sequence);
54                 current_sequence = FastaSequence();
55             }
56             current_sequence.header = line.substr(1);
57         }
58         else {
59             current_sequence.sequence += line;
60         }
61     }
62
63     if (!current_sequence.header.empty()) {
64         sequences.push_back(current_sequence);
65     }
66
67     return sequences;
68 }
69
```

```

70 AlignmentResult needle_wunsch(const string &seq1, const string &seq2, int match,
71   int mismatch, int gap, bool use_model_x) {
72   int n = seq1.size();
73   int m = seq2.size();
74
75   vector<vector<int>> score_matrix(n + 1, vector<int>(m + 1, 0));
76
77   for (int i = 0; i <= n; i++) {
78     score_matrix[i][0] = i * (use_model_x ? 1 : gap);
79   }
80   for (int j = 0; j <= m; j++) {
81     score_matrix[0][j] = j * (use_model_x ? 1 : gap);
82   }
83
84   for (int i = 1; i <= n; i++) {
85     for (int j = 1; j <= m; j++) {
86       int cost = use_model_x ? ((seq1[i-1] == 'X' || seq2[j-1] == 'X') ? 0 :
87         (seq1[i-1] == seq2[j-1] ? 0 : 1)) : (seq1[i-1] == seq2[j-1] ? match
88         : mismatch);
89
90       int score_diagonal = score_matrix[i - 1][j - 1] + cost;
91       int score_up = score_matrix[i - 1][j] + (use_model_x ? 1 : gap);
92       int score_left = score_matrix[i][j - 1] + (use_model_x ? 1 : gap);
93
94       score_matrix[i][j] = use_model_x ? min({score_diagonal, score_up,
95         score_left}) : max({score_diagonal, score_up, score_left});
96     }
97   }
98
99   string aligned_seq1, aligned_seq2;
100   int i = n, j = m;
101
102   while (i > 0 || j > 0) {
103     if (i > 0 && j > 0) {
104       int cost = use_model_x ? ((seq1[i-1] == 'X' || seq2[j-1] == 'X') ? 0 :
105         (seq1[i-1] == seq2[j-1] ? 0 : 1)) : (seq1[i-1] == seq2[j-1] ? match
106         : mismatch);
107
108       int score_diagonal = score_matrix[i - 1][j - 1] + cost;
109       int score_up = score_matrix[i - 1][j] + (use_model_x ? 1 : gap);
110       int score_left = score_matrix[i][j - 1] + (use_model_x ? 1 : gap);
111
112       if (score_matrix[i][j] == score_diagonal) {
113         aligned_seq1 += seq1[i - 1];
114         aligned_seq2 += seq2[j - 1];
115         i--;
116         j--;
117       }
118       else if (score_matrix[i][j] == score_up) {
119         aligned_seq1 += seq1[i - 1];
120         aligned_seq2 += '-';
121         i--;
122       }
123       else {
124         aligned_seq1 += '-';
125         aligned_seq2 += seq2[j - 1];
126         j--;
127       }
128     }
129     else if (i > 0) {
130       aligned_seq1 += seq1[i - 1];
131       aligned_seq2 += '-';
132       i--;
133     }
134     else if (j > 0) {
135       aligned_seq1 += '-';
136       aligned_seq2 += seq2[j - 1];
137       j--;
138     }
139   }
140   return AlignmentResult(aligned_seq1, aligned_seq2);
141 }

```



```

127     }
128     else {
129         aligned_seq1 += '-';
130         aligned_seq2 += seq2[j - 1];
131         j--;
132     }
133 }
134
135 reverse(aligned_seq1.begin(), aligned_seq1.end());
136 reverse(aligned_seq2.begin(), aligned_seq2.end());
137
138 return {aligned_seq1, aligned_seq2, score_matrix[n][m]};
139 }
140
141 vector<vector<double>> computeDistanceMatrix(const vector<FastaSequence> &sequences
142 ) {
143     int n = sequences.size();
144     vector<vector<double>> distance_matrix(n, vector<double>(n, 0.0));
145     vector<int> self_scores(n, 0);
146
147     for (int i = 0; i < n; i++) {
148         self_scores[i] = needle_wunsch(sequences[i].sequence, sequences[i].sequence
149             , MATCH, MISMATCH, GAP, false).score;
150     }
151
152     for (int i = 0; i < n; i++) {
153         for (int j = i + 1; j < n; j++) {
154             AlignmentResult result = needle_wunsch(sequences[i].sequence, sequences
155                 [j].sequence, MATCH, MISMATCH, GAP, false);
156             double distance = 1.0 - (static_cast<double>(result.score) / max(
157                 self_scores[i], self_scores[j]));
158
159             distance_matrix[i][j] = distance;
160             distance_matrix[j][i] = distance;
161         }
162     }
163
164     return distance_matrix;
165 }
166
167 vector<GuideTree> computeUPGMA(vector<vector<double>> &distance_matrix, int n) {
168     vector<GuideTree> guide_tree;
169     vector<int> cluster_sizes(n, 1);
170     vector<bool> active(n, true);
171
172     for (int step = 0; step < n - 1; step++) {
173         int min_i = -1, min_j = -1;
174         double min_distance = numeric_limits<double>::max();
175
176         for (int i = 0; i < n; i++) {
177             for (int j = i + 1; j < n; j++) {
178                 if (active[i] && active[j] && distance_matrix[i][j] < min_distance)
179                 {
180                     min_distance = distance_matrix[i][j];
181                     min_i = i;
182                     min_j = j;
183                 }
184             }
185         }
186
187         guide_tree.push_back({min_i, min_j});
188
189         for (int k = 0; k < n; k++) {

```

```

185         if (!active[k] || k == min_i || k == min_j) {
186             continue;
187         }
188         distance_matrix[min_i][k] = distance_matrix[k][min_i] =
189             (cluster_sizes[min_i] * distance_matrix[min_i][k] +
190              cluster_sizes[min_j] * distance_matrix[min_j][k]) /
191             (cluster_sizes[min_i] + cluster_sizes[min_j]);
192     }
193
194     cluster_sizes[min_i] += cluster_sizes[min_j];
195     active[min_j] = false;
196 }
197
198 return guide_tree;
199 }
200
201 vector<string> applyGaps(const vector<string> &group, const string &aligned_rap) {
202     vector<string> result;
203
204     for (const string &sequence : group) {
205         string new_sequence = "";
206         int sequence_index = 0;
207
208         for (char c : aligned_rap) {
209             if (c == '-') {
210                 new_sequence += '-';
211             }
212             else {
213                 new_sequence += sequence[sequence_index];
214                 sequence_index++;
215             }
216         }
217         result.push_back(new_sequence);
218     }
219
220     return result;
221 }
222
223 vector<string> mergeGroups(const vector<string> &group1, const vector<string> &
224     group2) {
225     string rep_a = group1[0];
226     string rep_b = group2[0];
227
228     AlignmentResult result = needle_wunsch(rep_a, rep_b, 0, 1, 1, true);
229
230     vector<string> aligned_group1 = applyGaps(group1, result.aligned_seq1);
231     vector<string> aligned_group2 = applyGaps(group2, result.aligned_seq2);
232
233     vector<string> merged;
234     merged.insert(merged.end(), aligned_group1.begin(), aligned_group1.end());
235     merged.insert(merged.end(), aligned_group2.begin(), aligned_group2.end());
236
237     return merged;
238 }
239
240 vector<string> performMSA(const vector<FastaSequence> &sequences, const vector<
241     GuideTree> &guide_tree) {
242     vector<vector<string>> groups;
243     for (const auto &seq : sequences) {
244         groups.push_back({seq.sequence});
245     }
246
247     for (const auto &merge : guide_tree) {

```

```

246     vector<string> merged_groups = mergeGroups(groups[merge.sequence_index1],
247         groups[merge.sequence_index2]);
248     groups[merge.sequence_index1] = merged_groups;
249     groups[merge.sequence_index2] = {};
250 }
251 for (const auto &group : groups) {
252     if (!group.empty()) {
253         return group;
254     }
255 }
256
257 return {};
258 }
259
260 vector<string> replaceX(const vector<string> &aligned_sequences) {
261     vector<string> result;
262     for (const string &seq : aligned_sequences) {
263         string clean = "";
264         for (char c : seq)
265             clean += (c == 'X') ? '-' : c;
266         result.push_back(clean);
267     }
268     return result;
269 }
270
271 int main(int argc, char *argv[]) {
272     if (argc < 3) {
273         cerr << "Usage: " << argv[0] << " <output.txt> <fasta_file1> ... <
274             fasta_fileN>" << endl;
275         return 1;
276     }
277
278     string output_filename = argv[1];
279     ofstream output_file(output_filename);
280     if (!output_file.is_open()) {
281         cerr << "Error opening output file: " << output_filename << endl;
282         return 1;
283     }
284
285     auto log = [&](const string &text) {
286         cout << text;
287         output_file << text;
288     };
289
290     vector<FastaSequence> sequences;
291     for (int i = 2; i < argc; i++) {
292         string file_sequence = argv[i];
293         vector<FastaSequence> file_sequences = readFileFasta(file_sequence);
294         if (file_sequences.empty()) {
295             cerr << "No sequences found in file: " << file_sequence << endl;
296             return 1;
297         }
298         sequences.insert(sequences.end(), file_sequences.begin(), file_sequences.
299             end());
300     }
301     log("Loaded " + to_string(sequences.size()) + " sequences.\n");
302
303     vector<vector<double>> distance_matrix = computeDistanceMatrix(sequences);
304
305     log("\n=== Initial Distance Matrix ===\n");
306     for (int i = 0; i < (int)sequences.size(); i++) {
307         for (int j = 0; j < (int)sequences.size(); j++) {

```

```

306         ostringstream oss;
307         oss << fixed << setprecision(4) << distance_matrix[i][j] << " ";
308         log(oss.str());
309     }
310     log("\n");
311 }
312
313 vector<vector<double>> dist_copy = distance_matrix;
314 vector<GuideTree> guide_tree = computeUPGMA(dist_copy, sequences.size());
315
316 log("\n=== UPGMA Guide Tree ===\n");
317 int n = sequences.size();
318 vector<vector<double>> dist_log = distance_matrix;
319 vector<int> cluster_sizes_log(n, 1);
320 vector<bool> active_log(n, true);
321
322 for (int step = 0; step < (int)guide_tree.size(); step++) {
323     int min_i = guide_tree[step].sequence_index1;
324     int min_j = guide_tree[step].sequence_index2;
325     double min_distance = dist_log[min_i][min_j];
326
327     ostringstream step_log;
328     step_log << "\nStep " << step + 1 << ": merge seq " << min_i
329         << " (" << sequences[min_i].header << ") with seq " << min_j
330         << " (" << sequences[min_j].header << ")"
331         << " | distance: " << fixed << setprecision(2) << min_distance <<
332         "\n";
333     log(step_log.str());
334
335     for (int k = 0; k < n; k++) {
336         if (!active_log[k] || k == min_i || k == min_j) continue;
337         dist_log[min_i][k] = dist_log[k][min_i] =
338             (cluster_sizes_log[min_i] * dist_log[min_i][k] +
339              cluster_sizes_log[min_j] * dist_log[min_j][k]) /
340             (cluster_sizes_log[min_i] + cluster_sizes_log[min_j]);
341     }
342     cluster_sizes_log[min_i] += cluster_sizes_log[min_j];
343     active_log[min_j] = false;
344
345     log("Updated Distance Matrix:\n");
346     for (int i = 0; i < n; i++) {
347         if (!active_log[i]) continue;
348         for (int j = 0; j < n; j++) {
349             if (!active_log[j]) continue;
350             ostringstream oss;
351             oss << fixed << setprecision(4) << dist_log[i][j] << " ";
352             log(oss.str());
353         }
354         log("\n");
355     }
356
357     vector<string> aligned_sequences = performMSA(sequences, guide_tree);
358     aligned_sequences = replaceX(aligned_sequences);
359
360     log("\n=== Final MSA ===\n");
361     for (size_t i = 0; i < aligned_sequences.size(); i++) {
362         log(">" + sequences[i].header + "\n");
363         log(aligned_sequences[i] + "\n");
364     }
365
366     output_file << "\nGuide Tree Data\n";
367     output_file << "Sequences\n";

```

```

368     for (int i = 0; i < (int)sequences.size(); i++) {
369         output_file << i << " " << sequences[i].header << "\n";
370     }
371     output_file << "Merges\n";
372     for (int step = 0; step < (int)guide_tree.size(); step++) {
373         int min_i = guide_tree[step].sequence_index1;
374         int min_j = guide_tree[step].sequence_index2;
375         double dist = dist_log[min_i][min_j];
376         output_file << min_i << " " << min_j << " " << fixed << setprecision(2) <<
            distance_matrix[min_i][min_j] << "\n";
377     }
378     output_file << "end\n";
379
380     output_file.close();
381     cout << "Alignment completed. Results saved to " << output_filename << endl;
382
383     return 0;
384 }

```

4.2. Código Python

Listing 11: Programa Python

```

1  import matplotlib.pyplot as plt
2  import matplotlib.patches as mpatches
3
4
5  def parse_results(filename):
6      sequences = {}
7      merges = []
8
9      with open(filename, 'r') as f:
10         lines = f.readlines()
11
12         i = 0
13         in_tree_section = False
14
15         while i < len(lines):
16             line = lines[i].strip()
17             line_upper = line.upper()
18
19             if "GUIDE TREE DATA" in line_upper:
20                 in_tree_section = True
21                 i += 1
22                 continue
23
24             if in_tree_section:
25                 if line_upper == "SEQUENCES":
26                     i += 1
27                     while i < len(lines):
28                         current = lines[i].strip().upper()
29                         if current == "MERGES":
30                             i -= 1
31                             break
32                         parts = lines[i].strip().split()
33                         if len(parts) >= 2:
34                             try:
35                                 sequences[int(parts[0])] = parts[1]
36                             except ValueError:
37                                 pass
38                     i += 1
39

```

```

40         elif line_upper == "MERGES":
41             i += 1
42             while i < len(lines):
43                 current = lines[i].strip().upper()
44                 if current == "END":
45                     break
46                 parts = lines[i].strip().split()
47                 if len(parts) == 3:
48                     try:
49                         merges.append((int(parts[0]), int(parts[1]), float(
50                             parts[2])))
51                     except ValueError:
52                         pass
53                 i += 1
54
55         elif line_upper == "END":
56             break
57
58     i += 1
59
60     return sequences, merges
61
62 def get_leaf_order(sequences, merges):
63     clusters = {idx: [idx] for idx in sequences.keys()}
64
65     for idx_a, idx_b, _ in merges:
66         merged = clusters[idx_a] + clusters[idx_b]
67         clusters[idx_a] = merged
68         clusters[idx_b] = []
69
70     for idx in sequences.keys():
71         if clusters[idx]:
72             return clusters[idx]
73     return list(sequences.keys())
74
75
76 def draw_tree(sequences, merges):
77     fig, ax = plt.subplots(figsize=(14, 9))
78     ax.set_title("UPGMA Guide Tree", fontsize=16, fontweight='bold', pad=20)
79     ax.axis('off')
80
81     n = len(sequences)
82     colors = ['#E63946', '#2A9D8F', '#E9C46A', '#F4A261', '#6A4C93', '#457B9D']
83
84     leaf_order = get_leaf_order(sequences, merges)
85     y_positions = {idx: (n - i) * 3.0 for i, idx in enumerate(leaf_order)}
86
87     all_dists = [d for _, _, d in merges]
88     min_d, max_d = min(all_dists), max(all_dists)
89
90     def norm(d):
91         if max_d == min_d:
92             return 0.7
93         return 0.5 + (d - min_d) / (max_d - min_d) * 0.45
94
95     LEAF_X = 0.32
96
97     for idx, name in sequences.items():
98         ax.text(LEAF_X - 0.01, y_positions[idx], name,
99               ha='right', va='center', fontsize=10,
100              bbox=dict(boxstyle='round,pad=0.4', facecolor='#AED6F1',
101                      edgecolor='#2E86C1', linewidth=1.5))

```

```

102 node_y = dict(y_positions)
103 node_x = {idx: LEAF_X for idx in sequences}
104
105
106 for step, (idx_a, idx_b, dist) in enumerate(merges):
107     color = colors[step % len(colors)]
108     x_merge = norm(dist)
109     y_a = node_y[idx_a]
110     y_b = node_y[idx_b]
111     x_a = node_x[idx_a]
112     x_b = node_x[idx_b]
113     y_new = (y_a + y_b) / 2.0
114
115     ax.plot([x_a, x_merge], [y_a, y_a], color=color, lw=2.5, solid_capstyle='
116         round')
117     ax.plot([x_b, x_merge], [y_b, y_b], color=color, lw=2.5, solid_capstyle='
118         round')
119     ax.plot([x_merge, x_merge], [y_b, y_a], color=color, lw=2.5, solid_capstyle
120         ='round')
121     ax.plot(x_merge, y_new, 'o', color=color, markersize=9, zorder=5,
122             markeredgecolor='white', markeredgewidth=1.5)
123     ax.text(x_merge + 0.012, y_new + 0.25,
124             f"S{step+1} d={dist:.2f}",
125             fontsize=8, color='white', ha='left', va='bottom', zorder=6,
126             bbox=dict(boxstyle='round,pad=0.3', facecolor=color,
127                     edgecolor='none', alpha=0.9))
128
129     node_y[idx_a] = y_new
130     node_x[idx_a] = x_merge
131
132 patches = [
133     mpatches.Patch(color=colors[i % len(colors)],
134                    label=f"S{i+1}: {sequences[merges[i][0]]} + {sequences[
135                        merges[i][1]]}")
136     for i in range(len(merges))
137 ]
138 ax.legend(handles=patches, loc='lower right', fontsize=9,
139          framealpha=0.9, title="Merge steps")
140
141 ax.set_xlim(0.0, 1.1)
142 ax.set_ylim(0.5, (n + 1) * 3.0)
143
144 plt.tight_layout()
145 plt.savefig("guide_tree.png", dpi=150, bbox_inches='tight')
146 plt.show()
147 print("Saved: guide_tree.png")
148
149 if __name__ == "__main__":
150     import sys
151     if len(sys.argv) < 2:
152         print("Usage: python GuideTree.py <results.txt>")
153         sys.exit(1)
154
155     sequences, merges = parse_results(sys.argv[1])
156     draw_tree(sequences, merges)

```